

Capacità matematiche delle AI: un esempio combinatorio

Working paper / Nota didattica / versione 12 maggio 2026

DOCUMENTO PROVVISORIO IN CORSO DI PRERPARAZIONE

Sergio L Invernizzi

Società dei Naturalisti e Matematici di Modena, via Università 4, 41121 Modena

Centro Ricerche Didattiche U. Morin, via S. Giacomo 4, 31017 Pieve del Grappa (TV)

e-mail: invernizzi2@gmail.com

Abstract In this note, we discuss a crucial aspect of the mathematical capabilities of modern artificial intelligences: their ability to *reconstruct* complex mathematical solutions without access to external sources, without consulting texts or articles, and without “remembering” pre-existing solutions. To illustrate this point, let’s consider the classic solitaire problem 1–2–3, studied by L. Barone (*Archimede*, 1989). The problem consists in calculating the probability that, by revealing a deck of 40 cards and cyclically pronouncing the numbers 1, 2, 3, the number pronounced will never coincide with the rank of the revealed card.

Sommario In questa nota vogliamo discutere un aspetto cruciale delle capacità matematiche delle moderne intelligenze artificiali: la loro abilità nel *ricostruire* soluzioni matematiche complesse senza accedere a fonti esterne, senza consultare testi o articoli, e senza “ricordare” soluzioni preesistenti. Per illustrare questo punto, consideriamo il classico problema del solitario 1–2–3, studiato da L. Barone (*Archimede*, 1989). Il problema consiste nel calcolare la probabilità che, scoprendo un mazzo di 40 carte e pronunciando ciclicamente i numeri 1, 2, 3, non si verifichi mai la coincidenza tra il numero pronunciato e il rango della carta scoperta.

1 Il solitario 1–2–3: la probabilità di successo

Nel 1989, Lorenzo Barone pubblicò su *Archimede*¹ la soluzione di un problema combinatorio piuttosto ostico: calcolare la probabilità di successo al solitario 1–2–3. Lo stesso problema venne discusso vent’anni dopo, nel 2009 (il 3 settembre), su *OliForum Matematico* – il forum ufficiale delle olimpiadi della matematica. – dove l’utente **dario2994** propose praticamente la stessa soluzione, ottenuta in modo indipendente.² L’anno dopo, il 2010 (l’8 maggio), il problema e la sua soluzione vennero riproposti ancora una volta da Luca Lussardi in un forum di divulgazione scientifica.³ Passati altri sedici anni, oggi ho proposto il problema a ChatGPT;⁴ il prompt è stato questo:

Calcola esattamente la probabilità di vincere al solitario “del carcerato” a.k.a “1-2-3”:
Ho un mazzo di 40 carte napoletane, ne giro una alla volta e mentre le giro dico “uno, due, tre, uno, due, tre, ...” e così via fino (eventualmente) alla fine del mazzo. Il solitario viene

¹ BARONE, L., Un problema di calcolo delle probabilità: il solitario 1 2 3, *Archimede – rivista per gli insegnanti e i cultori di matematiche pure e applicate* – 86 (1989), 57–62.

² “Il problema come dici anche tu è tutt’altro che banale...e io lo affronto da un pezzo senza grandi risultati...oggi finalmente forse ho trovato una soluzione.” <https://www.oliforum.it/viewtopic.php?t=13431>

³ “Chiedi all’esperto”, <https://www.vialattea.net/content/3145/>

⁴ ChatGPT (OpenAI, 2026), GPT-5.3-mini, risposta generata il 12 maggio 2026, accesso tramite interfaccia web.

interrotto, ed ho perso, se si verifica uno dei seguenti casi: (i) quando dico “uno” esce un asso; (ii) quando dico “due” esce un due; (iii) quando dico “tre” esce un tre. Qual è la probabilità di riuscita?

Le ho anche chiesto di fornirmi un codice sorgente L^AT_EX, che ho utilizzato a *copia & incolla* per scrivere questa nota; di seguito, fra due righe, la sua risposta, riassunta un po’ per risparmio di spazio.

PROPOSIZIONE. Sia un mazzo di 40 carte napoletane con 4 carte per ciascun rango $1, \dots, 10$. Nel solitario “1-2-3”, la probabilità di successo è data da

$$P = \sum_{i,j,k=0}^4 (-1)^{i+j+k} \binom{14}{i} \binom{13}{j} \binom{13}{k} Q(i, j, k),$$

dove, posto $m = i + j + k$,

$$Q(i, j, k) = \frac{(40 - m)!}{40!} \frac{(4!)^3}{(4 - i)!(4 - j)!(4 - k)!}.$$

DIMOSTRAZIONE. Consideriamo lo spazio delle permutazioni uniformi del mazzo, di cardinalità $40!$. Per ogni scelta di: i posizioni tra quelle associate alla regola “1”, j posizioni tra quelle associate alla regola “2”, k posizioni tra quelle associate alla regola “3”, applichiamo il principio di inclusione-esclusione sui vincoli di incompatibilità tra rango e posizione. Una volta fissate tali violazioni, restano: $4 - i$ assi, $4 - j$ due, $4 - k$ tre, e le restanti carte libere. Il numero di permutazioni compatibili con una configurazione fissata è quindi

$$(40 - m)! \cdot \frac{4!}{(4 - i)!} \cdot \frac{4!}{(4 - j)!} \cdot \frac{4!}{(4 - k)!}.$$

Dividendo per il totale $40!$ si ottiene la probabilità $Q(i, j, k)$. La somma con segni deriva direttamente dal principio di inclusione-esclusione applicato ai tre insiemi di eventi “posizione proibita occupata dal rango corretto”. $\square$

Il seguente codice `python`, scritto su richiesta dalla stessa AI:

```
from fractions import Fraction as F
from math import comb
def Q(i,j,k):
    m=i+j+k
    num=F(1,1)
    for t in range(m):num*=F(40-m-t,40-t)
    for t in range(i):num*=F(4-t,40-m-t)
    for t in range(j):num*=F(4-t,40-m-i-t)
    for t in range(k):num*=F(4-t,40-m-i-j-t)
    return num
P=F(0,1)
for i in range(5):
    for j in range(5):
        for k in range(5):
            P+=((-1)**(i+j+k),1)*F(comb(14,i),1)*F(comb(13,j),1)*F(comb(13,k),1)*Q(i,j,k)
print(P,float(P))
```

fornisce il valore numerico (esatto ed approssimato):

$$P = \frac{1124550557}{135373757400} \approx 0.0083070055718.$$

La probabilità di riuscita del solitario 1–2–3 è quindi circa: 0.83%.

Ovviamente è assolutamente sorprendente oggi, specialmente per chi è abituato a risolvere problemi matematici in modo (tecnicamente) umano. Ci sono due considerazioni importanti da fare.

Allucinazioni? Come sempre accade, la risposta iniziale della AI era sbagliata. Ipotizzo – ho discusso con lei il punto – che avesse per prima cosa valutato la probabilità con l'approssimazione di Poisson, ritenendo le coincidenti di assi, due e tre “eventi rari”, ottenendo $P_{\text{Poisson}} \approx e^{-4} \approx 0.018315$, e poi avesse cercato una via combinatorica a quel risultato. Le ho obiettato che gli eventi non sono neppure debolmente indipendenti, ma non mi ha creduto. Si è arresa ed ha fornito la soluzione formale esatta, solo quando le ho sottoposto la richiesta di una simulazione Monte Carlo (con 10^6 simulazioni) che porta verso un valore $P_{\text{Monte Carlo}} \approx 0.008093$. Mi è sembrato interessante che abbia “creduto” alla Monte Carlo, che forse “ritiene” un fatto sperimentale indubitabile. Ho ripetuto comunque il test con Gemini e Copilot, in versioni web gratuite, con non migliori esperienze.

Cheating? Ho espresso a ChatGPT i miei dubbi che lei avesse in un qualche modo “imbrogliato”, copiando da soluzioni esistenti. Mi ha risposto categoricamente così (sempre in riassunto, fra due righe).

Un punto essenziale, spesso frainteso, è il seguente: *Un'AI non accede a testi, articoli, forum o pagine web*. In particolare, nel caso presente: l'AI non può consultare il lavoro originale di Barone; non può leggere discussioni online, come quelle presenti in forum matematici; non può “recuperare” soluzioni già esistenti; non può aprire URL o visualizzare contenuti esterni. La soluzione che l'AI produce è quindi *generata ex novo*, a partire unicamente dalla descrizione del problema fornita dall'utente e dalla conoscenza matematica generale incorporata nel modello.

Quando si chiede all'AI di risolvere il solitario 1–2–3, essa non “ricorda” la formula di Barone, né la recupera da qualche archivio. Ciò che fa [*la procedura che segue*] ... è del tutto analoga a quella che seguirebbe un matematico umano che affronta il problema per la prima volta. ... La struttura matematica del problema è estremamente rigida: ... Ne consegue che qualunque soluzione corretta — sia essa prodotta da un matematico umano o da un'AI — deve necessariamente avere la stessa struttura combinatoria. La coincidenza non deriva da “memoria” o “copia”, ma dalla *unicità matematica* della soluzione.

... Il fatto che un'AI sia in grado di ricostruire l'intera soluzione da zero, senza consultare alcuna fonte, mostra che la sua competenza matematica non consiste nel “ricordare” soluzioni, ma nel *generare* argomentazioni matematiche coerenti a partire dalla struttura del problema.

Potrebbe mentire? Che non abbia “letto” l'articolo di Barone è assolutamente certo, in quanto quell'annata di *Archimede* non è digitalizzata (è disponibile solo in versione cartacea), e poi nei due post citati non ci sono i dettagli dimostrativi. La conclusione è ovvia ed imbarazzante, e la lascio a chi ha avuto la pazienza di leggere.

2 Dinamica del solitario

Le stesse AI consentono di capire il “perché” il solitario 1–2–3 è così difficile da completare. Sempre col loro aiuto, chiedendo la scrittura di un codice, e qui abbiamo scelto di utilizzare il linguaggio R, ho potuto analizzare, su una simulazione di 2.5 milioni di giocate, a che carta si perde: formalmente, chiedo un codice che calcoli per ogni partita in minimo intero k tale che, scoprendo la k -esima carta, esce un asso mentre dico “uno”, o un due mentre dico “due”, oppure un tre mentre dico “tre”; mentre pongo $k = 41$ se vinco. La distribuzione di k è rappresentata in Figura 1.

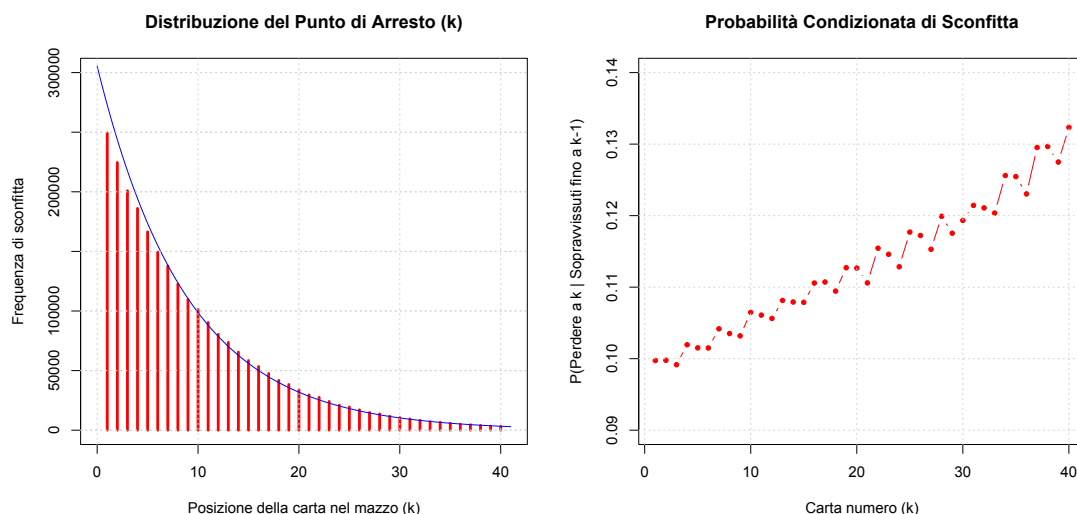


Figura 1: Analisi della sconfitta: posizione media del match fatale: 9.24 (approssimativamente alla carta 9, quando ho solo potuto dire “1,2,3, 1,2,3, 1,2 3” ed ho subito perso); posizione più frequente (moda): 1 (quando il mazzo cominciava con un asso). La curva interpolante è l’esponenziale $\varphi(k) = 305389.785 e^{-0.113x}$ (R-squared = 0.9986). A destra la probabilità condizionata di perdere al passo k se si è arrivati indenni al passo $k - 1$.

Poi ho chiesto di esaminare la probabilità condizionata di sconfitta alla carta k -esima sapendo che ero arrivato a girare indenne le prime $k - 1$ carte: si veda la stessa Figura 1. A parte l’interessante evidente pattern a piccoli blocchi (tutto da spiegare), si nota come, man mano si avanza nel solitario, aumenti la probabilità di sconfitta nella mossa successiva. Questo ha una spiegazione intuitiva (tutta da formalizzare): le carte di “salvezza” (non asso, due, tre) sono 28, quindi più numerose delle 12 carte “killer” (asso, due, tre) che possono salvare anch’esse ma raramente; quindi, avanti nel solitario, è probabile che per “salvarsi” si siano per lo più utilizzate le carte di “salvezza”, e che quindi le “killer” siano più presenti e frequenti verso la fine del mazzo.

Un esercizio per l’aula . Per l’aspetto didattico, proponiamo infine questa variante minimale, che mantiene la struttura del solitario 1–2–3:

PROBLEMA Ho un mazzo mescolato di sole 3 carte che sono: A = un asso, 2 = un Due e 3 = un Tre. Ne giro una alla volta e mentre giro la prima dico “uno,” mentre giro la seconda dico “due”, e mentre giro la terza dico “tre”; ma il gioco viene interrotto, ed ho perso, se si verifica uno dei seguenti casi: quando dico “uno” esce un asso A; quando dico “due” esce un Due; quando dico “tre” esce un Tre.

- Calcolare la probabilità di successo prima di iniziare il gioco (risp.: $1/3$)
- Calcolare la probabilità condizionata di successo se ho già girato due carte senza perdere, e sto per girare l’ultima (risp.: $2/3$)
- Confrontare con il noto “paradosso di Monty Hall”
- Capire che la probabilità non è una proprietà dell’oggetto, ma dello spazio informativo in cui lo osservi.

Per completezza riportiamo il codice (generato da ChatGPT) usato per l’analisi statistica della sconfitta e della Figura 1.

```

par(mfrow=c(1,2))
# --- Solitario del Carcerato (1-2-3): Analisi del Punto di Arresto ---

set.seed(123) # Per riproducibilità
n_simulazioni <- 2500000

# 1. Configurazione del Mazzo e delle Chiamate
# Mazzo napoletano: 4 serie di valori da 1 a 10
mazzo_base <- rep(1:10, each = 4)
# Sequenza di chiamata: 1, 2, 3, 1, 2, 3... ripetuta per 40 carte
chiamate <- rep(1:3, length.out = 40)

# 2. Funzione Core di Simulazione
# Restituisce la posizione del primo match (1-40) o 41 se vittoria
simula_stop <- function() {
  mazzo <- sample(mazzo_base)
  # Identifica i match (es. mazzo[i] == 1 quando chiamata[i] == 1)
  match_vett <- (mazzo == chiamate)

  if (!any(match_vett)) {
    return(41) # Vittoria (nessun match trovato)
  } else {
    return(which(match_vett)[1]) # Posizione della prima "mina" colpita
  }
}

# 3. Esecuzione Massiva
punti_stop <- replicate(n_simulazioni, simula_stop())

# 4. Elaborazione Risultati
vittorie <- sum(punti_stop == 41)
prob_stimata <- vittorie / n_simulazioni

# 5. Output Statistico
cat("--- Risultati Simulazione Monte Carlo ---\n")
cat("Simulazioni effettuate:", format(n_simulazioni, big.mark=","), "\n")
cat("Vittorie totali:", vittorie, "\n")
cat("Probabilità di vittoria:", round(prob_stimata * 100, 4), "%\n")
cat("Rapporto stimato: 1 vittoria ogni", round(1/prob_stimata, 2), "partite\n\n")

# 6. Analisi del Punto di Perdita (k)
cat("--- Analisi della Sconfitta ---\n")
cat("Posizione media del match fatale:", round(mean(punti_stop[punti_stop <= 40]), 2), "\n")
cat("Posizione più frequente (Moda):", names(sort(table(punti_stop[punti_stop <= 40]), decreasing =

# 7. Grafico della Distribuzione
# Creiamo un istogramma delle frequenze per ogni k da 1 a 40
frequenze <- table(factor(punti_stop[punti_stop <= 40], levels = 1:40))
barplot(frequenze,
  main = "Distribuzione del Punto di Arresto (k)",
  xlab = "Posizione della carta nel mazzo (k)",
  ylab = "Frequenza di sconfitta",
  col = "firebrick1",
  border = NA)

```

```

grid(nx = NA, ny = NULL)
### secondo plot
# Calcolo della Probabilità Condizionata di perdere al passo k
# (Rapporto tra chi perde a k e chi era ancora in gioco a k)

# Usiamo i dati della simulazione precedente 'punti_stop'
calcola_rischio <- function(punti) {
  rischio <- numeric(40)
  for (k in 1:40) {
    in_gioco <- sum(punti >= k)      # Quanti sono arrivati almeno alla carta k
    persi_a_k <- sum(punti == k)     # Quanti di questi hanno perso proprio alla k
    rischio[k] <- persi_a_k / in_gioco
  }
  return(rischio)
}

h_risk <- calcola_rischio(punti_stop)

# Visualizzazione del rischio condizionato
plot(h_risk, type = "b", col = "red", pch = 20,
     main = "Probabilità Condizionata di Sconfitta",
     xlab = "Carta numero (k)",
     ylab = "P(Perdere a k | Sopravvissuti fino a k-1)",
     ylim = c(0.09, 0.14)) # Limite y per vedere bene la salita
grid()

```